

Price tracker: upgrade plan (multi-store, store-neutral)

Status: **DRAFT, NOT STARTED**. Rewritten 2026-09-17 after the store spike ([store-spike.md](#)).

This plan turns the Tesco Price Tracker into a **store-neutral** price tracker. Tesco and Auchan are equal stores, and each can be switched off without a redeploy. It also covers the BSc thesis gaps (several webshops, semantic search, tests, security, documentation). Each numbered step can be committed on its own.

1. Decisions

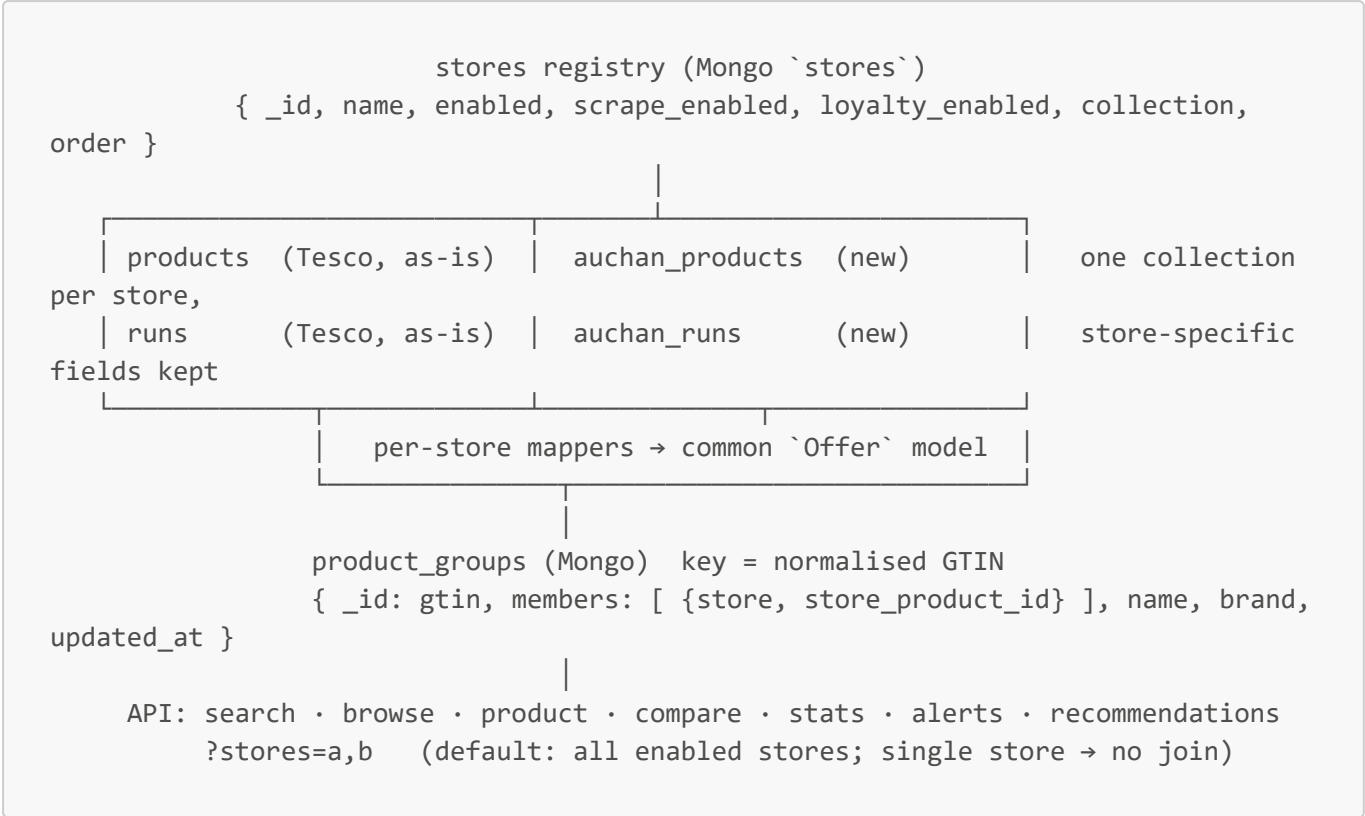
1.1 Made (2026-09-17)

Topic	Decision
Second store	Auchan (auchan.hu online shop). Penny, Lidl, SPAR, Kifli.hu rejected; see store-spike.md §7
Neutrality	No store is special. Tesco and Auchan are both entries in a store registry and can each be disabled for users
Naming	New neutral site name, hostname and API route. <code>/api/tesco/</code> and the current hostnames stay as working aliases
Default view	One product row with a price per enabled store. Barcode-linked products appear once; unlinked and own-brand products appear as single-store rows
Alerts	The user picks stores per alert (default: all enabled). The alert fires when any selected store meets the condition. Alerts on disabled stores are paused, not deleted
Order	Stores first, then search. Semantic search is built store-aware from the start
Storage	Separate collections per store; shared layers only join when a query needs it. The existing Tesco collection is not migrated
Loyalty prices	Modelled as a loyalty price channel for every store. Auchan loyalty prices come from a dedicated account, behind its own switch, as the last phase

1.2 Still open (resolve in Phase 0)

#	Question	Needed before
D1	Neutral site name, public hostname, API route prefix (proposal: <code>/api/prices/</code>)	Phase 6 (code can use neutral internal names earlier)
D2	Host CPU architecture for the embedding model image (compose defaults to <code>linux/amd64</code> ; the repo also has a Pi 5 ARM64 Qdrant build)	Phase 5
D3	Bizalomkártya programme terms allow the account's use	Phase 8

2. Target architecture



2.1 Identifiers

Thing	Format	Example
Store ID	lowercase slug	tesco, auchan
Offer reference (one listing in one store)	{store}:{store_product_id}	tesco:121262922, auchan:678170
Product group (linked by barcode)	g:{gtin}, GTIN digits with leading zeros stripped	g:54026193
Qdrant point ID	hash of the offer reference; existing Tesco points keep hash(tpnc) until re-vectorised	—

Barcode notes from the spike: Tesco stores GTIN-14 with leading zeros; Auchan has an EAN on every product; codes starting with 2 of length 13 are in-store weighed-item codes and are never linked across stores.

2.2 Common Offer model (API layer)

store, ref, store_product_id, gtin, group_id, name, brand, image_url, category_path[], pack_size, pack_unit, is_weighed, availability, prices { regular, promo, loyalty, unit_price, unit }, price_date, flags[], url (link to the store's own product page).

Price history uses one shape for every store: {date, regular, promo, loyalty}. Tesco's stored normal/discount/clubcard fields are mapped when read. Stored Tesco documents are not rewritten.

2.3 Store switches

Switch	Effect when false
enabled	Store hidden everywhere for users: search, browse, product pages, stats, recommendations, alert evaluation (alerts paused), store selector. Data is kept. Explicit ?stores= for it returns a clear error
scrape_enabled	Scheduler does not run the store's scraper; data stops updating
loyalty_enabled	Loyalty reader for the store is not run

The API caches the registry for ~60 s, so a toggle takes effect without a redeploy. If only one store is enabled, the frontend hides the selector, and the site stays store-neutral.

3. Phase 0: decisions and quick fixes

1. Resolve D1–D3 (§1.2).
2. Quick fixes that later phases rely on:
 - remove the API-key prefix from logs in `config.py`,
 - constant-time token check in `recomendation-system/app.py` `verify_api_token` (`hmac.compare_digest`),
 - fix the two stale recommendation test files (they import removed functions `get_product_vectors`, `sort_by_discount`) and add `recomendation-system/tests` to `pytest.ini` `testpaths`,
 - investigate why Tesco `deposit_amount` is always `null`.

4. Phase 1: store layer (no visible change)

Goal: the API works through the store registry and the **Offer** model with Tesco as the only store. Users see no difference.

1. **stores** collection, seeded idempotently at startup (Tesco **enabled: true**, Auchan **enabled: false**, **scrape_enabled: false**). Registry module with TTL cache, shared by `backend-api`, `alert-service`, `scheduler`.
2. Internal admin endpoint to toggle switches, behind the existing internal token and **not** routed through the public gateway.
3. **stores/** package with a **StoreMapper** interface (document → **Offer**, history → common shape). **TescoMapper** wraps the existing fields.
4. Normalised **gtin_norm** field + index on Tesco **products**: backfill script (idempotent, batch updates), and set on every scraper write.
5. `GET /api/v1/stores` → enabled stores with display data.
6. New store-neutral endpoints alongside the existing ones: `GET /api/v1/offers/{ref}`, `GET /api/v1/offers/{ref}/history`, `GET /api/v1/search?q=&stores=`, `GET /api/v1/browse?stores=`. Existing `/api/v1/products/*` endpoints keep their responses unchanged (extension, Blazor project).
7. Tests: registry cache and toggles, Tesco mapper, **stores** parameter validation, legacy endpoint responses unchanged.

5. Phase 2: Auchan ingestion

Goal: Auchan data collected daily into its own collections, monitored like Tesco. Not visible to users yet (`enabled: false`).

API facts (store-spike.md §7.6–7.7):

- token: `POST https://auchan.hu/fe-api/get-token {"grant_type":"anonymous"}`, 24 h,
- list: `GET /api/v2/products?page=&itemsPerPage=100&isCached=true&categories={id}` (max 100),
- tree: `GET /api/v2/tree/0?depth=1`; grocery top-level IDs 14479, 14602, 14656, 14740, 14830, 13307, 14863, 12617,
- details:
`/api/v2/products/{id}/variants/{variantId}/details[/description|ingredients|nutrition|parameterList]`,
- one national catalogue (delivery area does not change price or assortment),
- ~15k products, full crawl ~8 min at 1 request/s, 0 errors on two days, 0 EAN or ID changes day over day, ~7.5% price changes per day.

Steps:

1. `scraper/stores/auchan/` client: token fetch/refresh on 401, retries with backoff, 1 request/s, descriptive logs using the existing structured logging (`auchan.fetch`, `auchan.rate_limit`, `auchan.token_refresh`).
2. Crawl job: category IDs from configuration (with the tree endpoint used to detect new or removed categories), paged listing, de-duplication by product ID.
3. Write model in `auchan_products` (`_id` = Auchan product ID): name, brand, EAN + `gtin_norm`, category path, pack size/unit, weighed flag, image, flags, availability, `price_history[]` daily entries `{date, regular, promo, loyalty: null, unit_price, unit}, first_seen, last_seen`. Weighed items (`loose: true`) store the per-kg unit price.
4. Detail fetch only for new products or changed names (description, ingredients) to feed embeddings; low rate, separate step.
5. Run state in `auchan_runs` with resume, same pattern as the Tesco scraper.
6. Scheduler: per-store schedules and run states; each store runs only when `scrape_enabled`. A failure or rate limit in one store never blocks another.
7. Compose: `auchan-scraper` service (same image, own command), `vector.enable` label, health check. SecretManager: new configuration variables (`AUCHAN_CATEGORY_IDS`, `schedule`) in **both** `deployments.yaml` blocks.
8. Monitoring: Grafana alert rules mirroring the Tesco ones (incomplete, fatal, stale, upstream failure burst), with store-neutral titles.
9. Tests: recorded JSON fixtures for list, tree, token, 401 refresh, mapper, weighed item, run resume. No live network in CI.
10. Run it in production for at least 3 days with `enabled: false`, then check completeness and stability before Phase 3.

6. Phase 3: linking and shared queries

1. `product_groups` job after each store's run: group all offers by `gtin_norm` (excluding in-store weighed codes); upsert members; remove stale members. Log counts (`groups.linked`, `groups.single_store`).
 2. Search and browse (text search for now):
 - one store in `stores` → query that store's collection only,
 - several stores → query each enabled store's collection in parallel, map to `Offer`, merge by `group_id` (fallback: `ref`), rank (text score, then exact-name and barcode hits), paginate after merging.
 - Mongo text index on `auchan_products.name` (+ brand) like Tesco.
 3. `GET /api/v1/groups/{group_id}` → product with every enabled store's current offer. `GET /api/v1/groups/{group_id}/history` → aligned price history per store (the compare chart).
 4. Statistics: every existing `/stats/*` endpoint takes `stores`; the cache key includes the store set. New cross-store stats on linked groups:
 - share of linked products where each store is cheapest,
 - price index of a fixed basket of linked products per store over time,
 - average price difference per category. Products with a loyalty offer but no loyalty price are excluded from "cheapest store" statistics.
 5. Tests: merge and pagination, single-store short-circuit, disabled store excluded, stats cache key per store set, weighed codes never linked.
-

7. Phase 4: store-aware alerts and recommendations

1. Alert model: `target = {kind: "offer", ref}` or `{kind: "group", group_id}`, plus `stores: [...]` (default: all enabled at creation), existing type fields.
 2. Migration of existing alerts (idempotent script, dry-run first): `productId` → `target {kind: "offer", ref: "tesco:{productId}"}`, `stores: ["tesco"]`. Keep `productId` until the old API is retired.
 3. Scraper → alert-service trigger payload carries `ref`, `store`, `gtin_norm`. The evaluator matches offer alerts by `ref` and group alerts by group membership, only for stores that are both selected and enabled. Duplicate-send protection becomes per store run (`runKey = {store}:{date}`).
 4. Digest email shows store name and links to the store-neutral product page.
 5. Recommendations: candidate products limited to the requested/enabled stores; results de-duplicated by group; cold-start "best deals" merged across stores.
 6. Qdrant payload gains `store`, `ref`, `gtin_norm`; filters on `store`; payload indexes created. Auchan products are vectorised with the existing worker until Phase 5 moves vectorisation into the cluster.
 7. Tests: migration dry-run and idempotency, evaluator with store subsets and disabled stores, digest grouping, recommendation de-duplication.
-

8. Phase 5: semantic and hybrid search (store-aware)

1. `embedding-service` container: FastAPI + CPU `sentence-transformers`, model `intfloat/multilingual-e5-small` baked into the image, endpoint `POST /embed {texts, mode: query|passage}` (the service adds the E5 prefixes), internal network only, internal token (`hmac.compare_digest`), same logging format. Image platform from D2.
2. In-cluster vectorisation replaces the laptop worker for normal operation: after each store run, embed new or changed offers (`passage` mode) and upsert into Qdrant with `store/ref/gtin_norm/category` payload. Keep `worker.py` as an optional bulk backfill tool.

3. `GET /api/v1/search?q=&stores=&mode=text|semantic|hybrid` (default `hybrid`): Mongo text results + Qdrant results (filtered by stores) fused with Reciprocal Rank Fusion, then merged by group as in Phase 3. If the embedding service or Qdrant fails, text results are returned and a warning is logged.
4. `GET /api/v1/offers/{ref}/similar` and group-level "similar products" (Qdrant query by point, excluding the same group).
5. Cross-store matching for products **without** a shared barcode (optional, thesis material): embedding similarity + brand + normalised pack size → `product_matches` with confidence. Barcode groups are the labelled set for measuring precision and recall.
6. Search evaluation: ~30 labelled Hungarian queries; precision@10 and MRR for text / semantic / hybrid; results table in `docs/`.
7. Tests: prefixing, fusion, fallback path, store filter in Qdrant queries.

9. Phase 6: store-neutral frontend and branding

1. Apply the neutral name (D1): titles, SEO metadata, footer, privacy policy, manifest, translations (`hu.json`, `en.json`). No store is named in site-wide text.
2. Store selector: global chips from `GET /api/v1/stores`, remembered per visitor, overridable per search; hidden if only one store is enabled. Every product-list request sends `stores`.
3. Product rows: one row per group with each enabled store's price, cheapest highlighted; single-store rows show the store badge.
4. Product page (group): compare table (price, unit price, promo, loyalty, availability, link to the store) and a combined history chart per store.
5. Alerts UI: store checkboxes when creating (default: all enabled), paused badge when all selected stores are disabled.
6. Statistics page: store selector and the cross-store charts from Phase 3.
7. Routes: store-neutral product URLs (`/p/{group_id}`, `/o/{ref}`), with redirects from the old `/products/{tpnc}` URLs.
8. Tests: selector behaviour, hidden selector with one store, compare table, alert store selection, legacy URL redirects.

10. Phase 7: ecosystem and routing

Area	Repo	Change
Gateway routes	<code>Gavaller_websites_backend_ecosystem</code>	New <code>/api/{prefix}/*</code> → <code>api</code> <code>/api/v1/*</code> ; keep <code>/api/tesco/*</code> as alias; add the new hostname to the frontend route; keep the old hostnames
Frontend hostname / tunnel	<code>DockerNetworkArchitecture</code> / Cloudflare	New public hostname (D1)
Keycloak redirect URIs, CORS allowlists, auth-gateway return hosts	this repo, SecretManager	Add the new hostname

Area	Repo	Change
SecretManager	SecretManager	New variables in both blocks; controller redeploy after image tag changes (reconcile alone does not re-pull)
Grafana	Observability	Folder and dashboard titles store-neutral; store variable on product dashboards; Auchan alert rules
RefDataSync + ClickHouse dictionary	Gavaller_websites_backend_ecosystem , ClickHouseInfra	Product dimension keyed by ref with a store column; Tesco rows keep tpnc for existing panels
Browser extension	this repo	Stays a Tesco-site integration; keeps legacy routes; opens the neutral site
TescoDotnetPort (coursework)	TescoDotnetPort	No change; uses /api/tesco/ alias

11. Phase 8: Auchan loyalty prices (optional, behind [loyalty_enabled](#))

Facts: the product variant carries [loyaltyPrice](#), [loyaltyUnitPrice](#), [loyaltyPricePerKg](#), shown when [isLoyaltyPriceValid](#); anonymous responses omit them. The [Bizalomkártyás](#) flag marks affected products (380–613 at a time).

1. Check the programme terms (D3).
2. Account credentials and tokens are stored only in Infisical through SecretManager and injected as environment variables. Never commit them, never log them, never print them.
3. The login itself is not automated. Log in once by hand, store the refresh token, and let the reader keep the session alive with [grant_type=refresh_token](#). If refreshing fails, the reader stops, logs [auchan.loyalty_session_expired](#), and a Grafana alert asks for a manual re-login.
4. The reader only fetches flagged products and only writes the [loyalty](#) channel; normal prices keep coming from the anonymous crawl.
5. Validation before switching on: loyalty prices must equal the Auchan [LOYALTY](#) prices reported to GVH Árfigyelő for the overlapping products. If they differ (for example by loyalty level), do not publish them.
6. Until this phase is live, flagged products show a "card price available" badge and are excluded from cheapest-store statistics.

12. Phase 9: security hardening

1. Least-privilege MongoDB users per service (read-only for [api](#) on store collections; write only where needed); stop passing root credentials to application containers.

2. CORS allowlists instead of * (`ALLOWED_ORIGINS`, `ALERTS_ALLOWED_ORIGINS`) including the new hostname.
 3. Trust-boundary document: networks per service, which token protects which endpoint, why internal HTTP is acceptable or where TLS is added.
 4. Auth realms: document the two-realm flow (browser JWTs from `backend-ecosystem`, stack realm for auth-gateway and admin sync) with a sequence diagram, or consolidate.
 5. `mongo-express`: admin ingress only, strong basic-auth, or disabled in production.
 6. Tesco scrape reliability: pacing and thread count against the recurring 429 responses; several complete days before the presentation.
-

13. Tests (across phases)

Each phase lists its own tests. In addition:

1. CI integration job: `docker compose` with `mongo`, `qdrant`, `api`, `alert-service`, a stub `embedding-service`, seeded fixtures for **two stores**; checks search/browse/compare with every store combination, store toggles, alert create → trigger → digest (email stubbed), recommendations.
 2. Angular specs for services, the store selector, compare table, alerts (only `alerts.spec.ts` exists today).
 3. Optional Playwright smoke test: search → group page → compare → create alert.
 4. Coverage reports as CI artifacts (numbers for the thesis).
-

14. Documentation

1. Root `README.md`: what the system does, services, local run, deployment.
 2. `docs/architecture.md`: component diagram including the ecosystem (Cloudflare Tunnel → YARP gateway → services, Keycloak, Vector → ClickHouse → Grafana, SecretManager, Portainer), store registry and switches, daily scrape → link → vectorise → alert flow, search pipeline, auth sequence.
 3. `docs/stores.md`: how to add a store (mapper, scraper, registry entry, monitoring, tests), using Auchan as the worked example.
 4. `docs/deployment.md`: CI, GHCR images, Portainer, SecretManager variables, rollback.
 5. Decision records (`docs/adr/`): per-store collections + group layer, barcode linking, store switches, E5 + Qdrant, RRF hybrid search, loyalty reader design.
 6. Clean-up: `respond.json`, `versions/*.zip`, `.envbeforethe update`, `.env.prodversion`, empty `templates/`.
-

15. Milestones

Milestone	Phases	Result
M0	0	Decisions made; quick fixes; all tests run in CI
M1	1	Store registry and neutral API, Tesco only, no visible change
M2	2	Auchan collected daily and monitored, hidden from users
M3	3–4	Linked products, shared search/browse/stats, store-aware alerts and recommendations

Milestone	Phases	Result
M4	5	Hybrid semantic search across stores, no laptop dependency
M5	6–7	Store-neutral site live under the new name; old routes still work
M6	8	Auchan loyalty prices (if D3 allows)
M7	9, 13, 14	Hardened, tested, documented; ready for the thesis

Deploy note: Portainer pulls images from GHCR. When an image tag changes, a controller reconcile alone does not pull the new image; redeploy the stack.